Practice Set: Pure Virtual Functions, Abstract Classes, and Interface Architecture

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define a **Pure Virtual Function** and an **Abstract Class** in C++. Explain the pure specifier syntax (= 0) used during function prototype declaration.

Question 2: Explain why software engineering architectures require **Abstract Classes**. Provide a conceptual real-world example illustrating why direct instantiation of a generalized base entity makes no sense.

Question 3: Categorize operations on abstract classes into **Permitted Operations** and **Strictly Forbidden Operations** in C++.

Question 4: Compare a **Regular Concrete Class**, an **Abstract Class**, and a **Pure Interface Class** across direct instantiation capabilities, pure virtual methods, member variables (state), and primary architectural purpose.

Question 5: Explain how the compiler resolves a pure virtual function inside a class **Virtual Table (vtable)** for an abstract base class versus a concrete derived class.

Question 6: What causes the severe runtime crash error pure virtual function call (or __cxa_pure_virtual)? Explain the execution context under which this occurs.

Question 7: Explain why derived classes inherited from an abstract base class must override **all** pure virtual functions to become concrete classes. What happens if a derived class leaves even one pure virtual function un-implemented?

Question 8: Can a pure virtual function in C++ have a function body/definition? Explain how and where this is defined and discuss its typical architectural use case (e.g., pure virtual destructors).

Question 9: Outline the three essential **Modern C++ Best Practices** for working with pure virtual functions and abstract classes (focusing on C++11 override, virtual destructors, and dynamic cleanup).

Question 10: Write a complete C++ program demonstrating an abstract base class A containing a pure virtual function void show() = 0;. Implement a derived concrete class B that overrides show(), and in main(), invoke show() using an abstract base class pointer (A* ptr).

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): What is the exact syntax assigned to a virtual function declaration to make it a pure virtual function?

Question 12 (Viva): Is direct object creation (e.g., A obj;) permitted for an abstract class?

Question 13 (Viva): Can you create pointers or references of an abstract class type in C++?

Question 14 (Viva): Does C++ have a built-in interface keyword like Java or C#? How are interfaces constructed in C++?

Question 15 (Viva): What initial value or pointer handler does the compiler place in the vtable slot of an abstract base class for a pure virtual function?

Question 16 (Viva): What error occurs at compile time if a derived class fails to override a pure virtual function inherited from its parent abstract class?

Question 17 (Viva): Is a pure interface class allowed to contain member variables or state?

Question 18 (Viva): What keyword introduced in C++11 should always tag overridden pure virtual methods in derived classes to catch signature typos at compile time?

Question 19 (Viva): Why is a virtual destructor (virtual ~Base() = default;) strictly required in an abstract base class when deleting derived objects via base pointers?

Question 20 (Viva): What happens if a pure virtual function is invoked indirectly during the construction or destruction phase of an abstract base class?

Answers & Explanations

Answer: 01

Pure Virtual Function: A member function declared inside a base class assigned = 0 that provides no implementation within that base class. It acts as an absolute contract requiring all derived classes to override and implement it.

Abstract Class: Any class containing at least one pure virtual function automatically becomes an Abstract Class.

Pure Specifier Syntax (= 0):

virtual return_type function_name(parameters) = 0;

Answer: 02

Architectural Purpose: In software engineering, base classes frequently represent generalized concepts rather than concrete objects. Abstract classes enforce a strict object-oriented hierarchy by serving as uniform blueprints without allowing incomplete entities to be instantiated.

Real-World Example: Creating a generic Shape object directly makes no logical sense because a abstract "Shape" lacks concrete dimensions or form unless specified as a Circle, Rectangle, or Triangle.

Answer: 03

Permitted Operations

Declaring pointers to abstract classes: A* ptr;

Declaring references to abstract classes: A& ref = objB;

Inheriting from an abstract base class into derived child classes

Strictly Forbidden Operations:

Direct object creation: A obj; (Causes Compile Error).

Dynamic memory allocation via new: new A(); (Causes Compile Error).

Passing abstract classes by value in function parameters.

Answer: 04

Feature	**Regular Concrete Class**	**Abstract Class**	**Pure Interface**
Direct Instantiation	Allowed (A obj;)	Forbidden	Forbidden
Pure Virtual Methods	None	At least 1 method	All methods are pure virtual
Member Variables	Allowed	Allowed	Disallowed / No state
Primary Purpose	Concrete object implementation	Partial code reusability &	Strict API Contract

Feature	Regular Concrete Class	Abstract Class	Pure Interface
		blueprint	definition

Answer: 05

Abstract Base Class vtable: The compiler reserves a slot in the base class vtable for the pure virtual function. Because no body exists in the base class, the compiler fills this slot with a null pointer or points it to a runtime exception handler like __cxa_pure_virtual.

Concrete Derived Class vtable: Once the derived class overrides the pure virtual function with a concrete definition, the valid function address (e.g., &B::fun1()) replaces the null pointer in the derived class vtable slot.

Answer: 06

Cause & Context: The runtime crash pure virtual function call (or __cxa_pure_virtual) occurs if a pure virtual function is invoked indirectly during the **construction or destruction** phase of an abstract base class.

Mechanism: During base class construction/destruction, the derived object sub-part has either not been constructed yet or has already been destroyed. Thus, the derived vtable slot is unpopulated, and calling the function attempts to execute the base null pointer slot, causing an immediate program abort.

Answer: 07

Requirement: Derived classes must override and implement **every single** pure virtual function inherited from an abstract base class to become concrete, instantiable classes.

Consequence: If a derived class leaves even one pure virtual function un-implemented, that derived class itself remains an abstract class, and the compiler strictly forbids creating instances of it.

Answer: 08

Yes, C++ permits pure virtual functions to have a definition/body, provided the function definition is written **outside** the class declaration body.

Syntax Example:

```
class Base
{
   public:
   virtual ~Base() = 0; // Pure virtual destructor declaration
```

```
};
Base::~Base() { /* Definition outside class body */ }
```

Use Case: Primarily used when creating pure virtual destructors to force a class to be abstract while ensuring base destructor cleanup executes when derived objects are freed.

Answer: 09

Explicit override Keyword (C++11): Always tag overridden implementations in derived classes with override so the compiler catches typos or parameter mismatches at compile time.

Virtual Destructors in Base Classes: Always provide a virtual ~Base() = default; destructor in abstract base classes to prevent memory resource leaks during dynamic pointer deletion.

Smart Pointers & Polymorphism: Use smart pointers (e.g., std::unique_ptr<Base>) for managing abstract class pointers to ensure automatic, exception-safe dynamic memory cleanup.

Answer: 10

```
#include <iostream>
using namespace std;
class A
{
  public:
   virtual void show() = 0;

    virtual ~A() = default;
};
class B : public A
{
  public:
    void show() override
    {
       cout << "Implementation of show() in Concrete Derived Class B" << endl;
    }
};
int main()
{
    // A obj; // ERROR: Cannot instantiate abstract class

    B objB;
    A* ptr = &objB; // Base class pointer pointing to derived object
```

```
    ptr->show(); // Dynamic binding calls B::show()

    return 0;
}
```

Answer: 11
Appending **= 0** to the virtual function prototype declaration (e.g., virtual void fun() = 0;).

Answer: 12
No, direct object creation of an abstract class is strictly forbidden by the compiler.

Answer: 13
Yes, declaring pointers (A* ptr;) and references (A& ref = obj;) to an abstract class is fully permitted.

Answer: 14
No; C++ constructs pure interfaces using abstract classes that contain only pure virtual functions and no data members.

Answer: 15
A **null pointer** or a reference to a runtime handler function such as **__cxa_pure_virtual**.

Answer: 16
A **compile-time error** stating that the derived class is an abstract class and cannot be instantiated.

Answer: 17
No; pure interface classes should have no member variables or internal state.

Answer: 18
The **override** specifier.

Answer: 19
To ensure the derived class destructor is called properly when freeing derived objects allocated on the heap via base class pointers, preventing memory leaks.

Answer: 20
The program crashes with a **pure virtual function call** (or __cxa_pure_virtual) runtime error.